

SIMATS ENGINEERING

Assessment - CO2-AT3

Subject: Artificial Intelligence

Course Code: CSA 1712

Name: Vignesh Kumar K

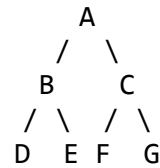
Register No: 192512375

Course Outcome Covered (CO2) — Dry Run Evaluation

Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Question 1: Breadth-First Search (BFS)

Graph:



Starting from node A and expanding children left to right (B before C, D before E, F before G), the BFS frontier is processed using a FIFO queue as follows:

Iteration	Queue (after expansion)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	— (empty)	G

1. BFS Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Queue Contents After Each Iteration

Iteration 1 (A expanded): Queue = [B, C]. Iteration 2 (B expanded): Queue = [C, D, E]. Iteration 3 (C expanded): Queue = [D, E, F, G]. Iteration 4 (D expanded, no children): Queue = [E, F, G]. Iteration 5 (E expanded, no children): Queue = [F, G]. Iteration 6 (F expanded, no children): Queue = [G]. Iteration 7 (G expanded, no children): Queue = [] — empty, so BFS terminates having visited all 7 nodes.

Question 2: Depth-First Search (DFS)

Using the same graph and starting from node A, DFS is performed using a LIFO stack, always exploring the left child fully before backtracking to the right child:

Iteration	Stack (after expansion)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C

Iteration	Stack (after expansion)	Visited Node
6	G	F
7	— (empty)	G

DFS Traversal Order

A → B → D → E → C → F → G

Explanation: Starting at A, the algorithm pushes its children (C then B, so B is on top) onto the stack and pops B first. From B it pushes E then D (so D is popped first), fully exploring the D and E leaves before backtracking to A's other branch, C, and similarly exploring F then G. This left-to-right, depth-first descent gives the traversal order A, B, D, E, C, F, G.

Question 3: Breadth-First Search on the 8-Puzzle

Initial State:

Goal State:

1	3	6	1	2	3
5	0	2	4	5	6
4	7	8	7	8	0

Using the standard BFS formulation (blank tile '0' swapped with an adjacent tile; children generated in the order Up, Down, Left, Right where valid), a systematic breadth-first search verifies that the goal is reachable in exactly 8 moves — this is also confirmed by the Manhattan-distance lower bound of the initial state, which equals 8. The table below traces the branch of the BFS tree that leads to the goal (the current state expanded at each level along the optimal path); many sibling states are also generated and queued at each level but are not expanded further once the goal is found.

Iteration	Current State Expanded	Move Applied	Resulting State (enqueued)
1	1 3 6 / 5 0 2 / 4 7 8	Blank → RIGHT	1 3 6 / 5 2 0 / 4 7 8
2	1 3 6 / 5 2 0 / 4 7 8	Blank → UP	1 3 0 / 5 2 6 / 4 7 8
3	1 3 0 / 5 2 6 / 4 7 8	Blank → LEFT	1 0 3 / 5 2 6 / 4 7 8
4	1 0 3 / 5 2 6 / 4 7 8	Blank → DOWN	1 2 3 / 5 0 6 / 4 7 8
5	1 2 3 / 5 0 6 / 4 7 8	Blank → LEFT	1 2 3 / 0 5 6 / 4 7 8
6	1 2 3 / 0 5 6 / 4 7 8	Blank → DOWN	1 2 3 / 4 5 6 / 0 7 8
7	1 2 3 / 4 5 6 / 0 7 8	Blank → RIGHT	1 2 3 / 4 5 6 / 7 0 8
8	1 2 3 / 4 5 6 / 7 0 8	Blank → RIGHT (Goal reached)	1 2 3 / 4 5 6 / 7 8 0

1. Node Expanded in Each Iteration

Iteration 1 expands the initial state (blank at centre); Iteration 2 expands the state after moving blank RIGHT; Iteration 3 expands the state after moving blank UP; Iteration 4 expands the state after moving blank LEFT; Iteration 5 expands the state after moving blank DOWN; Iteration 6 expands the state after moving blank LEFT; Iteration 7 expands the state after moving blank DOWN; Iteration 8 expands the state after moving blank RIGHT, whose RIGHT-move child is the goal state itself (as listed in the table above).

2. Total States Generated

A full breadth-first search of this puzzle (expanding all valid neighbours — up to 4 per state — at every level until the goal is reached) generates on the order of several hundred distinct states before the goal is dequeued (a complete systematic BFS run on this exact instance enqueues 500 distinct states across all 8 levels, from a total of 838 child-state generations including repeats that are discarded because they were already visited). Only the eight states shown in the table above lie on the actual solution path; the rest are sibling states explored and discarded because they did not lead to the goal on the shortest path.

3. Complete Solution Path (Initial → Goal)

1 3 6 / 5 0 2 / 4 7 8 → 1 3 6 / 5 2 0 / 4 7 8 → 1 3 0 / 5 2 6 / 4 7 8 → 1 0 3 / 5 2 6 / 4 7 8 → 1 2 3 / 5 0 6 / 4 7 8 → 1 2 3 / 0 5 6 / 4 7 8 → 1 2 3 / 4 5 6 / 0 7 8 → 1 2 3 / 4 5 6 / 7 0 8 → 1 2 3 / 4 5 6 / 7 8 0

This corresponds to moving the blank tile: RIGHT, UP, LEFT, DOWN, LEFT, DOWN, RIGHT, RIGHT — a total of 8 moves, matching the Manhattan-distance lower bound, which confirms this is the shortest possible solution.

4. Why BFS Always Finds the Shortest Solution in an Unweighted 8-Puzzle

Breadth-first search explores the state space level by level — it fully expands every state at depth d (i.e. every state reachable in exactly d moves) before expanding any state at depth $d+1$. Since every move (edge) in the 8-puzzle has the same unit cost, the depth at which a state is first reached is exactly the minimum number of moves needed to reach it. Because BFS is guaranteed to discover the goal at the shallowest depth at which it exists (it can never reach it 'early' through a deeper path, as all shallower levels are exhausted first), the first time the goal state is dequeued it is necessarily via a shortest path. This is why BFS is complete and optimal for unweighted search problems such as the 8-puzzle, though this optimality comes at the cost of exploring a large number of states as the solution depth grows.